



SH320 Data Engineering

Spring 2026 Semester

NumPy

② N-Dimensional Arrays

Hyun Ahn

School of Computing and Artificial Intelligence

hyunahn@hs.ac.kr

Core Libraries for Data Science



- **[W2] N-Dimensional Arrays**
 - NumPy arrays
 - Array manipulation
- **[W3] Efficient Computation in NumPy**
 - Universal functions
 - Advanced features

Data Engineering



2.1 NumPy 소개

Is Python Fast?

- 파이썬은 빠르다? : 코드 생산성이 높다
 - 인터프리터 방식
 - 동적타입 dynamic typing 언어
 - 간결한 문법 simplified syntax

정적타입	동적타입
C	Python
int a = 5;	a = 5

Java

```
public class JavaPyramid1 {
    public static void main(String[] args) {
        for(int i=1; i<=5; i++) {
            for(int j=0; j<i; j++) {
                System.out.print("*");
            }
            System.out.println("");
        }
    }
}
```

vs

Python

```
def create_pyramid(rows):
    for i in range(rows):
        print('*' * (i+1))
```

Is Python Slow?



- 파이썬은 느리다? : 반복문(for, while)의 느린 실행 속도
 - 파이썬은 동적타입 언어 ⇨ 타입 확인과 함수 디스패치 발생
 - 타입 확인 type checking : 실행 시점에 객체의 타입을 결정
 - 함수 디스패치 function dispatch : 객체 타입에 맞는 함수를 동적으로 할당
 - 반복문의 경우, 각 반복마다 타입 확인/함수 디스패치를 수행

Type Checking and Function Dispatch



1. Type Checking

- **동적 타이핑:** Python에서는 변수를 선언할 때 타입을 지정하지 않습니다. 타입은 값을 할당하는 시점에서 결정됩니다.

python

Copy code

```
x = 10      # x는 정수 타입
x = "hello" # x는 문자열 타입
```

- **타입 오류:** 실행 시간에만 타입 오류를 감지할 수 있습니다. 이는 코드의 안정성을 저해할 수 있습니다.

python

Copy code

```
def add(a, b):
    return a + b

result = add(10, "20") # TypeError 발생
```

- **타입 힌트:** Python 3.5 이후, 개발자들은 타입 힌트를 사용하여 변수나 함수의 예상 타입을 지정할 수 있습니다. 이는 실제 타입 체크를 하는 것은 아니지만, 타입 체크 도구나 IDE에서 코드를 검증하는 데 도움을 줍니다.

python

Copy code

```
def add(a: int, b: int) -> int:
    return a + b
```

2. Function Dispatch

- **동적 디스패치:** 객체의 메소드가 실행 시간에 결정됩니다. 이는 객체 지향 프로그래밍에서 다형성(polymorphism)을 구현하는 데 중요한 역할을 합니다.

python

Copy code

```
class Cat:
    def sound(self):
        return "Meow"

class Dog:
    def sound(self):
        return "Woof"

def animal_sound(animal):
    return animal.sound()

a = Cat()
print(animal_sound(a)) # 출력: Meow

b = Dog()
print(animal_sound(b)) # 출력: Woof
```

- **오버헤드:** 동적 디스패치는 메소드나 함수를 호출할 때 추가적인 오버헤드를 발생시킬 수 있습니다. 이는 프로그램의 성능에 영향을 미칠 수 있습니다.

결론적으로, Python의 동적 타이핑은 코드 작성의 유연성을 제공하지만, 실행 시간에 타입 오류를 감지하는 문제점이 있습니다. 이를 해결하기 위해 타입 힌트나 다양한 도구를 사용할 수 있습니다. 또한, 동적 디스패치를 통해 다형성을 구현할 수 있지만, 성능의 오버헤드를 고려해야 합니다.

ex) Reciprocal Computations

- 파이썬 반복문에서 100,000 개의 역수 계산 $\Rightarrow$ 약 1.47초 소요

```
[3]: def compute_reciprocals(values):  
      output = np.empty(len(values))  
      for i in range(len(values)): 반복문  
          output[i] = 1.0 / values[i]  
      return output
```

```
[4]: values = np.random.randint(1, 10, size=5)  
      print(values)
```

```
[6 1 4 4 8]
```

```
[5]: compute_reciprocals(values)
```

```
[5]: array([0.16666667, 1.          , 0.25        , 0.25        , 0.125        ])
```

```
[6]: """  
      Looping on a big array  
      """  
      big_array = np.random.randint(1, 100, size=1000000)  
      %timeit compute_reciprocals(big_array)
```

```
1.47 s  $\pm$  10.2 ms per loop (mean  $\pm$  std. dev. of 7 runs, 1 loop each)
```



참고:

Galaxy S21 연산 속도: 1720.3 **GFLOPS**
(1초 동안 1,720,300,000,000 개의 부동소수점 연산 수행)

- 수치연산에 특화된 라이브러리
 - 다차원 배열 n -dimensional array을 중심으로 한 빠르고, 다양한 수치계산 함수들을 제공
 - 많은 부분이 C 와 Fortran 으로 구현됨 $\Rightarrow$ 파이썬 기본 라이브러리보다 빠른 연산 가능
- 패키지 설치: `pip install numpy`
- 패키지 사용하기: `import numpy as np`

NumPy Array vs. Python List



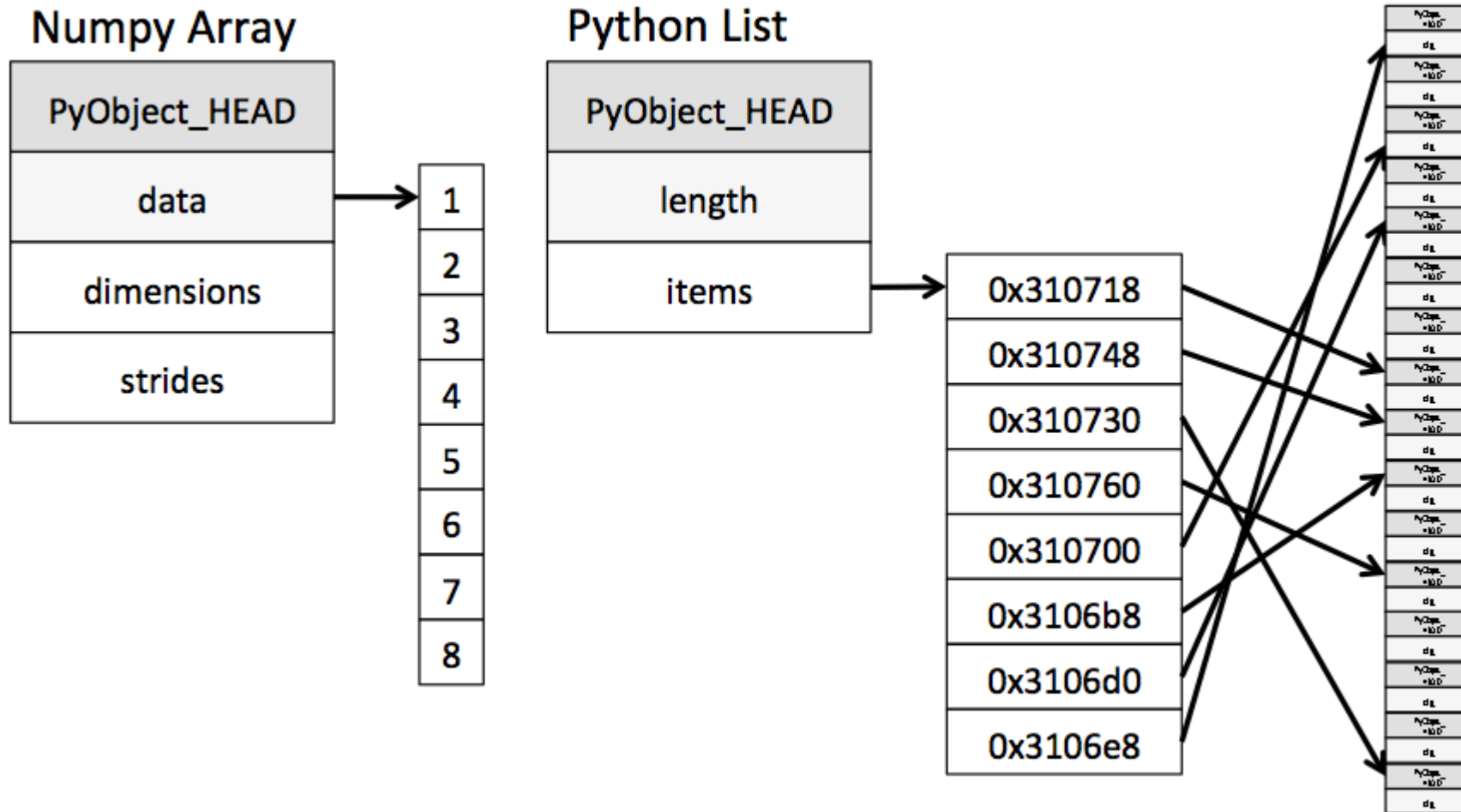
- **파이썬 리스트**

- 서로 다른 타입의 데이터 저장 가능
- 각 데이터 아이템은 객체

- **NumPy 배열**

- 서로 다른 타입의 데이터 저장 가능
- 단일 타입일 경우 고정된 타입의 배열로 처리 ➔ 리스트보다 빠름

NumPy Array vs. Python List



Creating a NumPy Array

- 리스트 객체로부터 생성

- `np.array(list)`

```
my_list = [1, 2, 3, 4, 5, 6]  
my_array = np.array(my_list)  
print(my_array)
```



```
[1 2 3 4 5 6]
```

- 리스트/NumPy 배열 동시에 생성

```
my_array = np.array([1, 2, 3, 4, 5, 6])
```

Operations with NumPy Arrays



- 리스트 컴프리헨션 기능은 편리하다.

```
prices = [30, 25, 40, 20, 20, 35, 50, 35]
new_prices = [price-5 for price in prices]
print(new_prices)
```



[25, 20, 35, 15, 15, 30, 45, 30]

- NumPy 배열 연산은 더 편리하다.

```
prices_arr = np.array(prices)
prices_arr -= 5
print(prices_arr)
```



[25 20 35 15 15 30 45 30]

Exercise: Operation with Test Scores



- 실습 문제: 시험 점수 처리

- 세 번의 시험에 대한 점수를 NumPy 배열로 선언하세요
(`test_1`, `test_2`, `test_3`)

	Tanya	Manual	Adwoa	Jeremy	Cody
test_1	92	94	88	91	87
test_2	79	100	86	93	91
test_3	87	85	72	90	92

Exercise: Operation with Test Scores



- 세 번째 시험(`test_3`) 채점에 문제가 발견되었습니다.
따라서, 모든 학생의 세 번째 시험에 2점을 더하여 새로운 배열 `test_3_fixed`에 저장하세요.

```
[92 94 88 91 87]  
[79 100 86 93 91]  
[89 87 74 92 94]
```

Exercise: Operation with Test Scores



- 학생 별 평균 시험 점수를 계산하기 위해 먼저 학생 별 전체 시험 점수 총합을 계산하여 배열 `total_grade` 에 저장하세요.
 - 세 번째 시험점수는 `test_3_fixed` 를 사용할 것

```
[260 281 248 276 272]
```

- 학생 별 평균 시험 점수를 계산하여 배열 `final_grade` 에 저장하세요.

```
[86 93 82 92 90]
```

Exercise: Operation with Test Scores



	Tanya	Manual	Adwoa	Jeremy	Cody
test_1	92	94	88	91	87
test_2	79	100	86	93	91
test_3	87	85	72	90	92

- Jeremy 의 두 번째 시험점수를 `jeremy_test_2` 에 저장하세요.
- Manual 과 Adwoa 의 첫 번째 시험점수를 `manual_adwoa_test_1` 에 저장하세요.

Exercise: Operation with Test Scores

- 전체 학생들의 점수를 2차원 배열 `student_scores` 에 저장하세요.
 - 2차원 배열 생성: `arr = np.array([[sub_arr1], [sub_arr2], ...])`
 - 2차원 배열 인덱싱: `arr[row_index, column_index]`

```
[[ 92  94  88  91  87]
 [ 79 100  86  93  91]
 [ 87  85  72  90  92]]
```

- 2차원 배열 `student_scores` 로부터 Tanya 의 세 번째 시험 점수를 구하세요.

```
87
```

- `student_scores` 로부터 Cody 의 전체 시험 점수를 구하세요.

```
[87 91 92]
```

Logical Operations with Arrays

- 배열 원소 단위의 논리 연산
 - 원소 별로 논리식에 대한 bool 값을 계산

```
a = np.array([10, 2, 2, 4, 5, 3, 9, 8, 9, 7])  
mask = a > 5  
print(mask)
```



```
[True False False False False False True True True True]
```

Logical Operations with Arrays

- 논리식을 만족하는 부분집합 구하기 = selection

```
selected = a[a > 5]  
print(selected)
```



[10 9 8 9 7]

- 논리식 결합: 논리합('|') 또는 논리곱('&') 사용
 - 각 논리식을 괄호 () 안에 작성

```
selected = a[(a > 5) | (a < 3)]  
print(selected)
```



[10 2 2 9 8 9 7]

Exercise: Goldilocks Porridge Festival



- 실습 문제: 골디락스 포리지(죽) 페스티벌
 - 페스티벌 아침에 만들어진 포리지의 온도를 바탕으로 평가
 - 각 포리지의 온도를 `porridge` 배열 저장하세요.

[79 65 50 63 56 90 85 98 79 51]

Exercise: Goldilocks Porridge Festival



- 온도가 60 도 미만인 포리지들은 먹기가 좀 차갑네요. 이들을 **cold** 배열에 저장하세요.

[50 56 51]

- 온도가 80 도 초과인 포리지들은 뜨거워서 먹기가 어렵네요.
이들을 **hot** 배열에 저장하세요.

[90 85 98]

- 온도가 60 도 이상이고 80 도 이하인 포리지들은 먹기가 아주 적당합니다. 이들을 **just_right** 배열에 저장하세요.

[79 65 63 79]

2.2 NumPy 배열 조작

NumPy Array: Basic Manipulations



- NumPy 배열 조작 유형
 - 배열 속성 지정: 배열의 크기, 형상^{shape}, 메모리 소비량, 데이터 타입을 결정
 - 배열 인덱싱: 배열의 개별 원소에 접근 및 값 할당
 - 배열 슬라이싱: 배열의 부분집합을 계산
 - 배열 재구조화: 배열의 형상을 변경
 - 배열 결합 및 분할

Creating an array

- `np.arange(start, stop*, step)` : 특정 범위의 정수 배열 생성
 - *start*: 범위 시작 값(포함), 디폴트 값: 0
 - *stop*: 범위 마지막 값(제외)
 - *step*: 간격, 디폴트 값: 1

```
np.arange(3)
```

[0]	[1]	[2]
0	1	2

```
np.arange(3, 9)
```

[0]	[1]	[2]	[3]	[4]	[5]
3	4	5	6	7	8

```
np.arange(3, 9, 2)
```

[0]	[1]	[2]
3	5	7

4 6 8

2 steps 2 steps

Creating an array

- 난수 random number 배열 생성하기
 - `np.random.seed(num)`: 똑같은 난수 배열 생성을 위한 시드 값
 - `np.random.randint(high, size)`: 정수로 구성된 난수 배열 생성
 - 참고([link](#))

```
# Seed for reproducibility
np.random.seed(0)
arr1 = np.random.randint(10, size=6) # 1D array
arr2 = np.random.randint(10, size=(3, 4)) # 2D array
arr3 = np.random.randint(10, size=(3, 4, 5)) # 3D array
```

NumPy Array Attributes



- NumPy 배열 속성
 - `arr.ndim`: 차원 개수
 - `arr.shape`: 배열 차원 크기
 - `arr.size`: 전체 배열 크기(배열 원소 개수)
 - `arr.dtype`: 배열 데이터 타입
 - `arr.itemsize`: 배열 원소 별 메모리 크기
 - `arr.nbytes`: 전체 배열 메모리 크기

Array Slicing: Accessing Subarrays

- 배열 **슬라이싱**: 배열의 부분집합을 선택

– *arr[start:stop:step]*

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
arr[3:7]	0	1	2	3	4	5	6	7	8	9

arr[:5]	0	1	2	3	4	5	6	7	8	9
---------	---	---	---	---	---	---	---	---	---	---

arr[5:]	0	1	2	3	4	5	6	7	8	9
---------	---	---	---	---	---	---	---	---	---	---

arr[::3]	0	1	2	3	4	5	6	7	8	9
----------	---	---	---	---	---	---	---	---	---	---

arr[2:8:2]	0	1	2	3	4	5	6	7	8	9
------------	---	---	---	---	---	---	---	---	---	---

Multi-dimensional Subarrays



- 다차원 배열 슬라이싱

- 예, 2차원 배열: `arr[start1:stop1:step1, start2:stop2:step2]`

`arr[3:5, 4:7]`

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	5	0	3	3	7	9	3	5
[1]	2	4	7	6	8	8	1	6
[2]	7	7	8	1	5	9	8	9
[3]	4	3	0	3	5	0	2	3
[4]	8	1	3	3	3	7	0	1
[5]	9	9	0	4	7	3	2	7

Multi-dimensional Subarrays

- 실습 문제

`arr[:2, :3]`

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	5	0	3	3	7	9	3	5
[1]	2	4	7	6	8	8	1	6
[2]	7	7	8	1	5	9	8	9
[3]	4	3	0	3	5	0	2	3
[4]	8	1	3	3	3	7	0	1
[5]	9	9	0	4	7	3	2	7

`arr[:3, ::2]`

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
[0]	5	0	3	3	7	9	3	5
[1]	2	4	7	6	8	8	1	6
[2]	7	7	8	1	5	9	8	9
[3]	4	3	0	3	5	0	2	3
[4]	8	1	3	3	3	7	0	1
[5]	9	9	0	4	7	3	2	7

Subarrays as Views

- 배열 슬라이스는 복사본^{copy}이 아닌 **뷰**^{view}

– 배열 슬라이스 값을 변경 ⇨ **원본 배열 값도 변경됨** (≠ 리스트)

```
arr_sub = arr[:2, :2]
```

	[0]	[1]	[2]	[3]
[0]	12	5	2	4
[1]	7	6	8	8
[2]	1	6	7	7

arr: 원본 배열

```
arr_sub[0, 0] = 99
```

	[0]	[1]	[2]	[3]
[0]	99	5	2	4
[1]	7	6	8	8
[2]	1	6	7	7

```
print(arr)
```

	[0]	[1]	[2]	[3]
[0]	99	5	2	4
[1]	7	6	8	8
[2]	1	6	7	7

Subarrays as Views

– `arr.copy()`: 배열(또는 슬라이스) 복사본 생성

```
arr_sub = arr[:2, :2].copy()
```

	[0]	[1]	[2]	[3]
[0]	12	5	2	4
[1]	7	6	8	8
[2]	1	6	7	7

arr: 원본 배열



	[0]	[1]
[0]	12	5
[1]	7	6

arr_sub:
배열 슬라이스 복사본

Reshaping of Arrays

- 배열의 **형상**^{shape} = 벡터/행렬의 **차원**^{dimension}
 - 배열의 형상을 변경(재구조화)하여 처리하는 경우가 많음
- `arr.reshape()`: 배열 형상 변경 메서드
 - 변경 전/후의 배열 크기가 동일해야 함

$arr \in \mathbb{R}^9$
 (row-vector)

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
1	2	3	4	5	6	7	8	9

`grid = arr.reshape(3, 3)`



$grid \in \mathbb{R}^{3 \times 3}$
 (2D matrix)

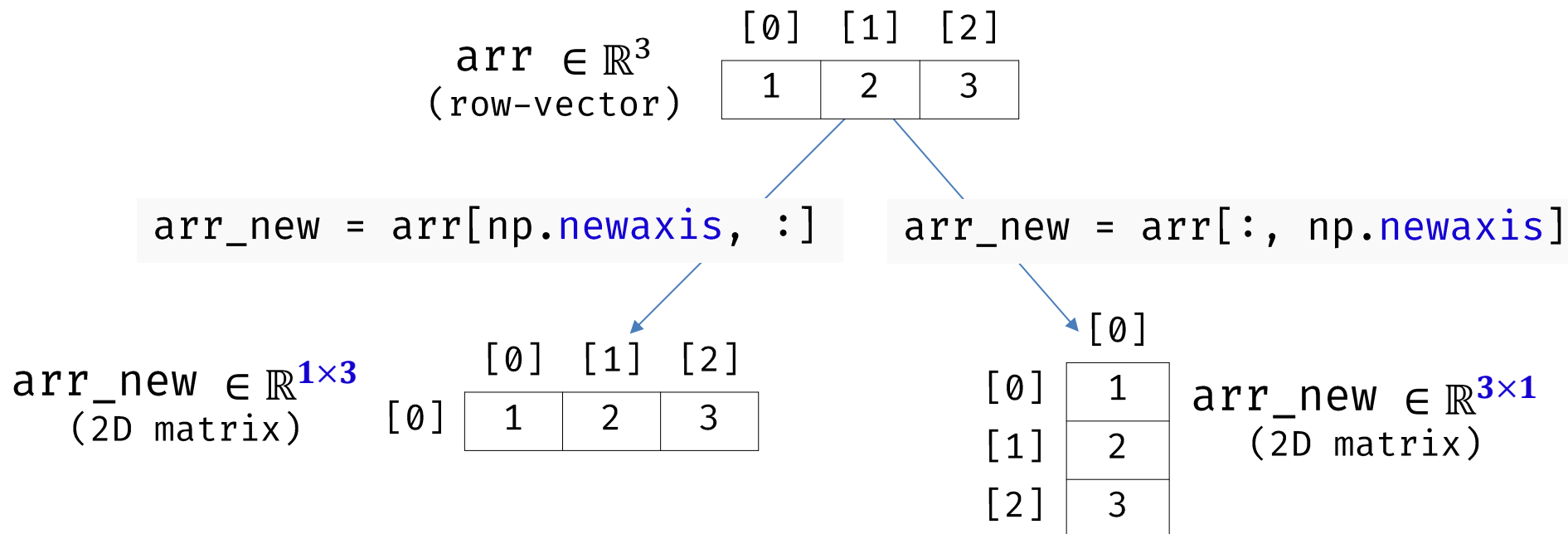
	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9

Reshaping of Arrays

- 재구조화의 흔한 패턴: **차원을 추가**

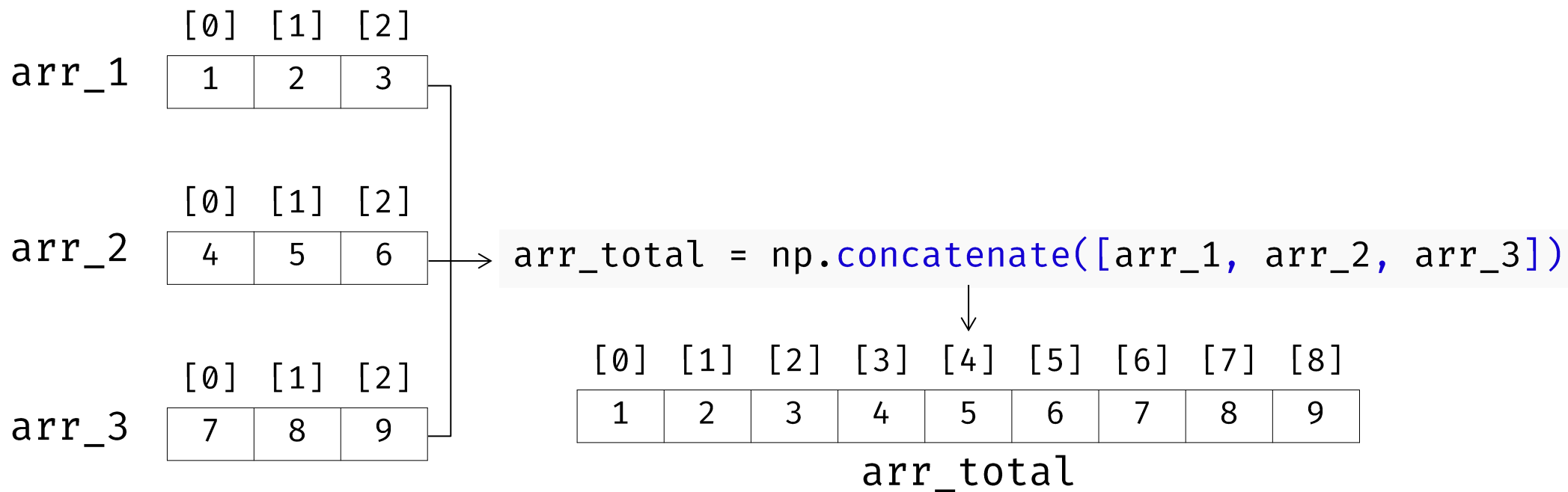
– 예: 1차원 배열^{row-vector} $\Rightarrow$ 2차원 행렬^{matrix}

- `np.newaxis`: 배열의 추가 차원



Array Concatenation

- 배열 연결: 두 개 이상의 배열을 하나로 결합
 - `np.concatenate()`: 배열 연결 메서드



Array Concatenation

- `np.vstack()`: **종**vertical **방향으로** 배열 연결
- `np.hstack()`: **횡**horizontal **방향으로** 배열 연결

arr_1

[0]	[1]	[2]
1	2	3

arr_2

[0]	[1]	[2]
4	5	6

arr_3

[0]	[1]	[2]
7	8	9

arr_total = np.vstack([arr_1, arr_2, arr_3])

↓

	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9

arr_total

Array Splitting

- **배열 분할:** 배열을 두 개 이상의 부분 배열로 나누기
 - `np.split()`, `np.vsplit()`, `np.hsplit()`: **배열 분할 메서드**

```
arr_1, arr_2, arr_3 = np.split(arr, [3, 5])
```

	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
arr	1	2	3	99	99	3	2	1



	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	1	2	3	99	99	3	2	1



	[0]	[1]	[2]
arr_1	1	2	3
arr_2	99	99	
arr_3	3	2	1



수고하셨습니다.